

Como esta organizado este proyecto

Respuesta corta

La estructura actual **tiene sentido**, pero mezcla dos formas de clasificar componentes:

1. Por su nivel de reutilizacion: **layout**, **sections** y **ui**.
2. Por la pagina a la que pertenecen: **home**, **about**, **contact**, **services**, etc.

Eso no es incorrecto. De hecho, es una estructura bastante comun. Lo que causa confusion es que algunos nombres no coinciden exactamente con las rutas y que dentro de **home** conviven secciones grandes con piezas pequenas que solo sirven a esas secciones.

La idea general se puede resumir asi:

```
pages      = decide que aparece en cada URL
layouts    = coloca el marco general de la pagina
sections   = construye bloques grandes de contenido
ui          = proporciona piezas pequenas y reutilizables
data        = guarda el contenido y los datos
assets      = guarda imagenes e iconos procesados por Astro
public      = guarda archivos publicos servidos directamente
lib         = guarda comportamiento compartido
```

Analogia principal: construir un edificio

Imagina que el sitio web es un edificio.

- **pages** es el plano que dice que habitaciones tiene cada apartamento.
- **layouts** es la estructura fija del edificio: entrada, pasillos, techo y salida.
- **sections** son habitaciones completas: sala, cocina, dormitorio y oficina.
- **ui** son elementos que se pueden usar en muchas habitaciones: puertas, sillas, lamparas y mesas.
- **data** es el inventario que indica que texto, precios o elementos se colocan dentro.
- **assets** es el almacen de materiales que Astro puede cortar, optimizar y preparar.
- **public** es una vitrina exterior: lo que dejas ahi se entrega directamente, sin procesamiento.
- **lib** contiene mecanismos compartidos, como el ascensor o el sistema electrico.

Una pagina no deberia fabricar cada silla y cada puerta. Su trabajo principal es elegir las habitaciones y colocarlas en orden.

El recorrido de una pagina

La pagina principal, **src/pages/index.astro**, contiene esto conceptualmente:

```
<SiteLayout>
  <Hero />
```

```
<Service />
<About />
<WhyChooseUs />
<Insights />
</SiteLayout>
```

Su responsabilidad es **componer** la pagina, no implementar todos sus detalles.

El flujo es:

```
URL /
-> src/pages/index.astro
    -> SiteLayout
        -> Header
        -> contenido de la pagina
        -> Footer
    -> sections/home/Hero.astro
    -> sections/home/Service.astro
    -> sections/home/About.astro
    -> sections/home/WhyChooseUs.astro
    -> sections/home/Insights.astro
```

Otra analogia: **index.astro** es el director de una obra de teatro. No interpreta todos los personajes; decide quienes salen y en que orden.

Que hace cada carpeta

src/pages

Astro convierte los archivos de esta carpeta en rutas del sitio.

src/pages/index.astro	-> /
src/pages/nosotros/index.astro	-> /nosotros
src/pages/contact/index.astro	-> /contact
src/pages/servicios/index.astro	-> /servicios
src/pages/servicios/[id].astro	-> /servicios/algun-id

[id].astro representa una ruta dinamica. Es como una plantilla capaz de producir diferentes paginas de servicio segun el **id** recibido.

Lo ideal es que estos archivos sean relativamente pequenos y se concentren en:

- Definir la URL.
- Establecer titulo y descripcion SEO.
- Obtener datos propios de la ruta, si hace falta.
- Importar y ordenar secciones.
- Envolver el contenido con el layout correcto.

Las paginas actuales siguen bastante bien esta regla. Por ejemplo, `contact/index.astro` solo configura el `SiteLayout` y coloca `ContactSection`.

src/layouts

Los layouts definen la envoltura de una pagina.

`SiteLayout.astro` hace esta composicion:

```
Layout
  Header
  contenido de la pagina mediante <slot />
  Footer
```

La analogia es un marco de fotografia: cambia la foto, pero el marco puede seguir siendo el mismo.

En este proyecto:

- `Layout.astro` contiene la base del documento HTML y la configuracion global.
- `SiteLayout.astro` agrega el encabezado y el pie de pagina del sitio.

src/components/layout

Aqui viven componentes visuales estructurales que aparecen en muchas paginas:

- `Header.astro`
- `Footer.astro`
- Navegacion movil y de escritorio.
- Columnas y datos del footer.

Aunque la palabra `layout` tambien aparece en `src/layouts`, no tienen exactamente la misma funcion:

- `src/layouts` contiene envolturas de pagina usadas directamente por las rutas.
- `src/components/layout` contiene las piezas que forman esas envolturas.

Siguiendo la analogia del edificio: el layout es el plano general del vestibulo; `Header` y `Footer` son las piezas concretas con las que se construye.

src/components/sections

Una `section` es un bloque grande y reconocible de una pagina. Normalmente puede responder a una pregunta como: "¿Que parte de la pagina estoy viendo?".

Ejemplos:

- El hero principal.
- La lista de servicios.
- Las estadisticas de la empresa.
- La llamada a la accion final.
- El formulario de contacto.

Las subcarpetas no representan otro tipo tecnico de componente. Solo indican **a que pagina o tema pertenece cada seccion:**

sections/home	= secciones y piezas propias de la portada
sections/about	= secciones de la pagina Nosotros
sections/contact	= contenido de Contacto
sections/services	= contenido de Servicios
sections/projects	= contenido de Proyectos
sections/shared	= secciones grandes usadas por varias paginas

Por tanto, **home**, **about** y **contact** no compiten con **sections**. La relacion es:

sections	<- que nivel de componente es
sections/home	<- en que pagina se utiliza
sections/about	<- en que pagina se utiliza
sections/contact	<- en que pagina se utiliza

Otra analogia: **sections** es un supermercado y **home**, **about** y **contact** son sus pasillos. Todos siguen siendo productos del supermercado; los pasillos solo ayudan a encontrarlos.

src/components/ui

ui contiene piezas visuales pequenas y reutilizables que no deberian conocer una pagina especifica.

Ejemplos actuales:

- **NumberedCard.astro**
- **FeatureCard.astro**
- **DetailCard.astro**
- **StatsGrid.astro**

Una buena pieza de **ui** recibe datos mediante propiedades y los representa. No deberia importar directamente todo el contenido de una pagina concreta ni decidir en que ruta aparece.

Por ejemplo, **CompanyGrid.astro** importa datos de **data/about** y utiliza **NumberedCard.astro**:

CompanyGrid	NumberedCard
-----	-----
Conoce la pagina	No conoce la pagina
Obtiene sus datos	Recibe title, description, etc.
Organiza varias tarjetas	Dibuja una tarjeta
Vive en sections/about	Vive en ui

Analogia: **CompanyGrid** es el encargado de organizar un comedor; **NumberedCard** es el modelo de silla que puede utilizar en ese comedor o en otro lugar.

src/data

Esta carpeta separa contenido y presentacion.

```
data/about.ts      = contenido de Nosotros
data/home.ts       = contenido de la portada
data/services.ts   = servicios disponibles
data/projects.ts   = proyectos
data/navigation.ts = enlaces de navegacion
```

Esto evita repetir grandes listas de textos dentro del HTML y permite recorrerlas con `.map()`.

Analogia: el componente es el menu impreso del restaurante y `data` es la lista de platos que el chef entrega para llenar ese menu.

src/assets y public

Ambas carpetas guardan archivos, pero no funcionan igual.

`src/assets` se usa cuando el archivo se importa desde el codigo:

```
import image from "@assets/images/example.avif";
```

Astro conoce ese archivo y puede incluirlo en su proceso de construccion.

`public` se usa mediante una URL directa:

```
<video src="/new/video.webm" />
```

El archivo se publica practicamente tal como esta.

Analogia:

- `src/assets` es material que pasa por el taller antes de instalarse.
- `public` es material terminado que se entrega directamente al visitante.

src/lib

Contiene logica reutilizable que no es un componente visual. En este proyecto incluye comportamiento para videos y animaciones al hacer scroll.

Es como la maquinaria que hace funcionar distintas partes del edificio sin ser una habitacion visible.

Por que `about`, `contact` y otros contienen codigo de grid

El nombre de una carpeta y el nombre de un archivo responden preguntas diferentes.

`components/sections/about/CompanyGrid.astro` significa:

- `components`: es una pieza de interfaz.
- `sections`: es una pieza relativamente grande de pagina.
- `about`: pertenece al tema o pagina Nosotros.
- `CompanyGrid`: presenta informacion de la empresa en forma de cuadrícula.

Que un archivo se llame `Grid` no significa que deba vivir en una carpeta global llamada `grid`. `Grid` describe **como organiza visualmente su contenido**; `about` describe **donde tiene sentido ese contenido**.

Analogia: una "estanteria de medicamentos" sigue perteneciendo a la farmacia. "Estanteria" describe su forma; "farmacia" describe su contexto.

La pregunta importante para decidir su ubicacion es:

Si elimino la pagina Nosotros, ¿este componente todavia tiene sentido en el proyecto?

- Si la respuesta es no, debe quedarse cerca de `about` o `nosotros`.
- Si la respuesta es si y varias paginas lo usan, probablemente pertenece a `shared` o `ui`.

Diferencia entre `sections/shared` y `ui`

Esta diferencia puede parecer pequena, pero es util:

`sections/shared`

Contiene bloques grandes reutilizados en varias paginas.

Ejemplos:

- `InteriorHero.astro`
- `PhotoCta.astro`

Pueden ocupar buena parte del ancho o alto de una pagina y representar una seccion completa.

`ui`

Contiene primitivas mas pequenas que se combinan para construir secciones.

Ejemplos:

- Una tarjeta.
- Un boton generico.
- Un bloque de estadística.
- Un campo de formulario reutilizable.

Analogia con LEGO:

- `ui` son bloques individuales.
- `sections/shared` son modulos ya armados que puedes poner en distintos modelos.
- `pages` son el modelo final y sus instrucciones de montaje.

Evaluacion de la estructura actual

Lo que esta bien

- Las rutas estan separadas de los componentes visuales.
- La portada solo compone sus secciones y es facil leer su orden.
- **Header** y **Footer** no se repiten en cada pagina.
- Existen componentes reutilizables en **ui**.
- Las secciones compartidas tienen una carpeta **shared**.
- Parte del contenido esta separado en **data**.
- Las paginas interiores reutilizan **InteriorHero** y **PhotoCta**.
- La estructura puede crecer sin obligar a poner todos los componentes en una sola carpeta.

Lo que genera confusion

1. La ruta se llama **nosotros**, pero sus componentes estan en **sections/about**.

Ambos nombres significan casi lo mismo, pero cambiar de idioma obliga a recordar una traduccion innecesaria.

2. Las carpetas no usan un criterio de idioma uniforme.

Hay rutas en espanol, como **nosotros**, **servicios** y **proyectos**, mientras los componentes usan **about**, **services** y **projects**. Esto no rompe el proyecto, pero reduce la facilidad para encontrar archivos.

3. **home** mezcla secciones grandes y componentes internos.

Hero.astro, **Service.astro** y **Insights.astro** parecen secciones principales. En cambio, **CarouselControls.astro**, **AccordionItem.astro**, **MetricCard.astro** y **CompanyCard.astro** son piezas internas de esas secciones. Tenerlas juntas funciona mientras haya pocos archivos, pero dificulta reconocer la jerarquia.

4. El nivel de abstraccion no es totalmente uniforme.

ContactSection.astro representa casi toda la pagina de contacto. En otras paginas, la ruta combina varias secciones separadas. No es un error, pero significa que el nombre **section** no siempre representa el mismo tamano.

5. Algunos nombres son demasiado generales.

Dentro de **home**, nombres como **About.astro** y **Service.astro** pueden confundirse con las paginas completas de Nosotros y Servicios. Algo como **HomeAbout.astro** no seria necesario porque la carpeta ya aporta contexto, pero **AboutIntro.astro** o **ServicesOverview.astro** explicarian mejor su responsabilidad.

Es la mejor estructura posible

No existe una unica "mejor estructura" para todos los proyectos. La mejor es la que permite responder rapido:

- ¿Donde esta el codigo de esta ruta?
- ¿Donde esta esta parte visual?
- ¿Es reutilizable o pertenece a una sola pagina?
- ¿Donde modifiko el contenido?

Para el tamaño actual del proyecto, la base es buena y **no conviene reemplazarla por una arquitectura complicada**. No hacen falta patrones empresariales, carpetas profundamente anidadas ni una carpeta distinta para cada tipo de CSS.

Si se quiere mejorar, bastaria con hacer consistentes los nombres y separar las piezas internas de las secciones de **home**.

Estructura recomendada para este proyecto

Una evolucion pequena y clara seria:

```
src/
├── assets/
│   ├── icons/
│   └── images/
├── components/
│   ├── layout/
│   │   ├── header/
│   │   ├── footer/
│   │   ├── Header.astro
│   │   └── Footer.astro
│   ├── ui/
│   │   ├── DetailCard.astro
│   │   ├── FeatureCard.astro
│   │   ├── NumberedCard.astro
│   │   └── StatsGrid.astro
│   └── sections/
│       ├── shared/
│       │   ├── InteriorHero.astro
│       │   └── PhotoCta.astro
│       ├── home/
│       │   ├── Hero.astro
│       │   ├── ServicesOverview.astro
│       │   ├── AboutIntro.astro
│       │   ├── WhyChooseUs.astro
│       │   ├── Insights.astro
│       │   └── components/
│       │       ├── CarouselControls.astro
│       │       ├── AccordionItem.astro
│       │       ├── MetricCard.astro
│       │       └── CompanyCard.astro
│       ├── nosotros/
│       │   ├── CompanyGrid.astro
│       │   └── ImpactGrid.astro
│       ├── contacto/
│       │   └── ContactSection.astro
│       └── servicios/
```



```
├── proyectos/
├── tecnologia/
├── data/
├── layouts/
├── lib/
├── pages/
└── styles/
```

La parte importante no es usar español específicamente. También sería correcto usar todo en inglés. Lo importante es escoger un idioma para nombres internos y mantenerlo de forma consistente.

Regla práctica para crear un componente nuevo

Usa estas preguntas en orden:

1. ¿Define una URL?

Va en **pages**.

Ejemplo: una nueva página **/empleos** sería **src/pages/empleos/index.astro**.

2. ¿Envuelve una página completa?

Va en **layouts**.

Ejemplo: un layout especial para páginas sin header.

3. ¿Es Header, Footer o navegación global?

Va en **components/layout**.

4. ¿Es un bloque grande de una página?

Va en **components/sections/<página>**.

Ejemplo: **components/sections/proyectos/ProjectGrid.astro**.

5. ¿Es un bloque grande usado en varias páginas?

Va en **components/sections/shared**.

Ejemplo: **PhotoCta.astro**.

6. ¿Es una pieza pequeña y reutilizable sin conocimiento de la página?

Va en **components/ui**.

Ejemplo: una tarjeta, un botón o una etiqueta.

7. ¿Solo ayuda internamente a una sección específica?

Debe vivir cerca de esa sección, por ejemplo en **sections/home/components**.

Ejemplo: controles que solo funcionan con el carrusel del hero de la portada.

Cuando mover algo a `ui`

No conviene mover un componente a `ui` solo porque podria reutilizarse algun dia.

Primero puede vivir junto a la seccion que lo necesita. Se mueve a `ui` cuando:

- Ya lo utilizan varias partes del sitio.
- Su API mediante props es generica.
- No importa datos propios de una pagina.
- No contiene textos o enlaces especificos de una ruta.

Analogia: no hace falta construir una bodega central para una silla que solo existe en una habitacion. Cuando muchas habitaciones usan el mismo modelo, entonces si tiene sentido almacenarlo como pieza compartida.

Ejemplo de responsabilidad correcta

Para una nueva seccion de testimonios en la portada:

```
data/home.ts
  -> guarda nombres, cargos y testimonios

components/ui/TestimonialCard.astro
  -> representa un testimonio generico

components/sections/home/Testimonials.astro
  -> importa los datos y organiza varias TestimonialCard

pages/index.astro
  -> decide en que posicion aparece <Testimonials />
```

Cada nivel responde a una pregunta distinta:

```
data      -> que contenido existe
ui        -> como se ve una unidad
section   -> como se organiza el bloque
page      -> donde aparece el bloque
layout    -> que estructura global lo rodea
```

Conclusion

La arquitectura actual no esta mal dividida. Utiliza una combinacion razonable de organizacion por capas y por pagina:

Por capas: pages -> sections -> ui
Por pagina: home, about, contact, services, projects...

El principal problema es de **consistencia y claridad de nombres**, no de arquitectura fundamental.

La recomendacion es conservar la base y aplicar mejoras pequenas:

- Usar el mismo idioma o vocabulario para rutas y carpetas relacionadas.
- Separar dentro de **home** las secciones principales de sus componentes auxiliares.
- Mantener las paginas como archivos de composicion pequenos.
- Mover a **ui** solamente las piezas verdaderamente reutilizables.
- Usar **shared** para secciones completas compartidas entre varias paginas.

En una frase: **las paginas arman, las secciones organizan y los componentes de UI resuelven las piezas pequenas.**